Topological Sort (medium)

`Topological Sort` is used to find a linear ordering of elements that have dependencies on each other. For example, if event 'B' is dependent on event 'A', 'A' comes before 'B' in topological ordering.

This pattern defines an easy way to understand the technique for performing topological sorting of a set of elements and then solves a few problems using it.

Let's see this pattern in action.

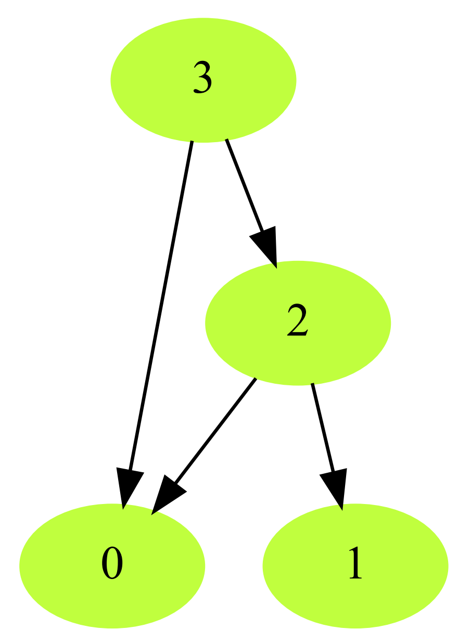
Problem Statement

Topological Sort of a directed graph (a graph with unidirectional edges) is a linear ordering of its vertices such that for every directed edge (U, V) from vertex U to vertex V, U comes before V in the ordering.

Given a directed graph, find the topological ordering of its vertices.

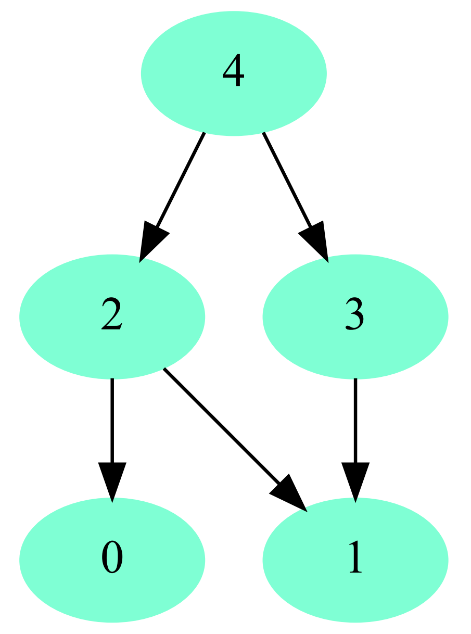
Example 1:

```
Input: Vertices=4, Edges=[3, 2], [3, 0], [2, 0], [2, 1]
Output: Following are the two valid topological sorts for the given
graph:
1) 3, 2, 0, 1
2) 3, 2, 1, 0
```



Example 2:

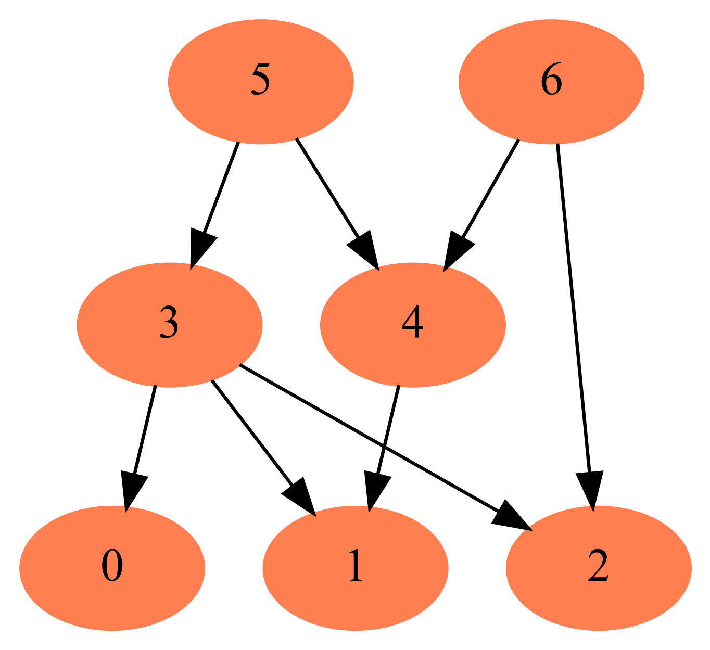
```
Input: Vertices=5, Edges=[4, 2], [4, 3], [2, 0], [2, 1], [3, 1]
Output: Following are all valid topological sorts for the given graph:
1) 4, 2, 3, 0, 1
2) 4, 3, 2, 0, 1
3) 4, 3, 2, 1, 0
4) 4, 2, 3, 1, 0
5) 4, 2, 0, 3, 1
```



Example 3:

```
Input: Vertices=7, Edges=[6, 4], [6, 2], [5, 3], [5, 4], [3, 0], [3, 1],
[3, 2], [4, 1]
Output: Following are all valid topological sorts for the given graph:
1) 5, 6, 3, 4, 0, 1, 2
2) 6, 5, 3, 4, 0, 1, 2
3) 5, 6, 4, 3, 0, 2, 1
4) 6, 5, 4, 3, 0, 1, 2
5) 5, 6, 3, 4, 0, 2, 1
6) 5, 6, 3, 4, 1, 2, 0

There are other valid topological ordering of the graph too.
```



Try it yourself

Try solving this question here:

Java

```
import java.util.*;

class Solution {
  public List<Integer> sort(int vertices, int[][] edges) {
    List<Integer> result = new ArrayList<>();
    // TODO: Write your code here
    return result;
  }
}
```

Test Case | Run Code Result | Stdout

```
4
[[3,2],[3,0],[2,0],[2,1]]
```

Use Example Testcases | Run | Submit

← Previous
Introduction To Topological Sort Pattern

Next →
Solution: Topological Sort

☐ Mark as Completed